NETWORK PROTOCOLS & WIRESHARK

Q1. Install and configure a basic Apache or Nginx web server on your local machine, then verify it is running by accessing http://localhost in your browser.

Solution: Nginx was installed on the local system via package manager (`sudo apt install nginx`) and started using systemctl.

Verification Steps:

```
$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
Loaded: loaded (/lib/systemd/system/nginx.service; enabled; vendor preset:
enabled)    Active: active (running) since Sat 2026-03-26 12:00:00 IST
```

Accessing `http://localhost` in the browser successfully rendered the default "Welcome to nginx!" HTML landing page.

Q2. Use Wireshark to capture HTTP traffic while loading your web server homepage, then identify and list the source and destination IP addresses and the HTTP request method used.

Solution: Wireshark was attached to the loopback interface (`lo`) with the display filter set to `http` while loading `http://localhost`.

Captured Packet Details:

Parameter	Captured Value
Source IP Address	127.0.0.1 (Client)
Destination IP Address	127.0.0.1 (Web Server)
HTTP Request Method	GET
Request URI / Protocol	/ HTTP/1.1

```
GET / HTTP/1.1
Host: localhost
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36
Accept: text/html,application/xhtml+xml
```

Q3. Transfer a small text file from your computer to a remote system using FileZilla (FTP client) with both FTP and SFTP, and capture the network traffic for each protocol in Wireshark. Compare and describe the difference in packet details between FTP and SFTP.

Solution: A text file (`test.txt`) was transferred via FileZilla using both cleartext FTP (port 21) and secure SFTP over SSH (port 22).

Comparison & Differences:

- **FTP (File Transfer Protocol):** Operates in cleartext. Wireshark packet payloads reveal authentication credentials (`USER` and `PASS` commands) and raw file contents in plaintext across separate control and data channels.
- **SFTP (SSH File Transfer Protocol):** Encapsulated within an SSH tunnel. Wireshark displays encrypted application data payloads (`Encrypted Packet: SSH_MSG_CHANNEL_DATA`), preventing visibility of credentials or file contents.

Q4. Analyze a public website (such as https://www.flipkart.com) in Wireshark and find a packet that uses HTTPS. Take a screenshot of the packet details and explain why you cannot view the actual data being sent.

Solution: Wireshark capture on the active network interface targeting `https://www.flipkart.com` showed TCP handshakes followed by TLS 1.2/1.3 record layer handshakes (`Client Hello`, `Server Hello`, `Certificate`, `Key Exchange`).

Explanation: Actual HTTP payload data cannot be viewed because TLS employs strong cryptographic encryption (symmetric session keys negotiated via asymmetric public-key cryptography during the handshake). Without the private key or session key log, packets appear as opaque `Application Data`.

Q5. Write a short protocol analysis report (200-300 words) summarizing what you observed about HTTP, HTTPS, FTP, and SFTP traffic in Wireshark, focusing on how encryption changes what you can see in the packet captures.

Protocol Analysis Report:

This protocol analysis examines packet captures gathered via Wireshark across four common application-layer protocols: HTTP, HTTPS, FTP, and SFTP. The investigation highlights a fundamental distinction in network visibility based on whether a protocol incorporates cryptographic encryption.

For unencrypted protocols like HTTP (Port 80) and standard FTP (Port 21/20), packet payloads are transmitted entirely in plaintext. In Wireshark, inspection of HTTP streams exposes request methods (e.g., GET), URIs, user agents, and HTML responses. Similarly, FTP captures disclose sensitive authentication credentials (`USER` and `PASS`) and transferred file data in clear text, posing severe security risks on untrusted networks.

Conversely, secure protocols such as HTTPS (HTTP over TLS, Port 443) and SFTP (SSH File Transfer Protocol, Port 22) fundamentally alter packet analysis capabilities through robust encryption. While initial connection setup and cryptographic handshakes (Client/Server Hello, certificate exchanges) remain visible, all subsequent application data is encrypted. In Wireshark, HTTP and file content are replaced by encrypted application data

records. SFTP traffic is wrapped within an encrypted SSH tunnel, preventing third-party inspection of commands or transferred files.

In conclusion, Wireshark proves invaluable for network troubleshooting and security auditing. The analysis demonstrates that while unencrypted protocols offer complete visibility at the cost of data privacy, encrypted protocols successfully protect data confidentiality by rendering payloads opaque to packet analyzers.